

TensorPool 3D Tier Partitioning

RTL Hierarchy Analysis and Implementation Assessment

Design target: TensorPool (`config=tenorpool`), MemPool repository, branch `tenorpool`

Physical-design target: Synopsys 3DIC Compiler, two-tier face-to-face stack

Abstract

This report reconstructs the TensorPool RTL hierarchy from the source, quantifies every candidate tier-crossing interface using widths extracted by elaboration, and evaluates four two-tier partitioning strategies against those numbers. The principal finding is that the inter-Group TCDM interconnect — twelve 16x16 crossbars that are, contrary to the module names, instantiated **inside** `mempool_group` rather than at cluster level — can be relocated to a second tier at a cost of **99,072** vertical connections, which is within 0.8% of the **98,304** wires that already have to cross the die horizontally today. The crossbar is width-preserving, and approximately 242,000 flip-flops of pipeline already sit exactly at the proposed cut, so the partition adds neither interface signals nor latency. Relocating the L1 TCDM instead would cost between 1.6x and 5x more vertical connections and would break the single-cycle local bank path that defines the MemPool memory model. The recommendation is to move the inter-Group interconnect, the cluster pipeline registers and the L2 subsystem to the top tier, and to keep L1 and compute on the bottom tier. Synopsys 3DIC Compiler cannot perform this partition: it assembles dies that are already implemented separately, so the split must be expressed explicitly in the RTL hierarchy.

Contents

1	Scope and Methodology	3
1.1	Objective	3
1.2	Method	3
1.3	Configuration under analysis	3
2	TensorPool RTL Hierarchy and Communication Structure	4
2.1	Where the crossbars actually are	4
2.2	Hierarchy tree	4
2.3	Interconnect inventory	5
2.4	Interface widths	5
2.5	Existing pipelining on the inter-Group path	5
2.6	AXI, DMA and L2 path	6
3	Candidate Tier Partitions	6
3.1	Tier-crossing signal counts	6
3.2	Option A — SubGroup-level folding	6
3.3	Option B — Group-level folding	7
3.4	Option C — Interconnect-centric partition	7
3.5	Option D — Memory plus interconnect partition	7
3.6	Comparison	7
4	Recommended Partition	8

4.1	Bond density feasibility	8
4.2	Prior art in the repository	8
5	Memory on the Top Tier Versus Interconnect on the Top Tier	9
5.1	The case against relocating L1	9
5.2	Where a memory partition could work	9
5.3	Filling the top tier without moving L1	9
5.4	Thermal considerations	10
6	Required RTL Restructuring	10
6.1	Option C	10
6.2	Option C+ (additional)	11
6.3	Options A and D (additional)	11
6.4	Reducing the pipeline depth	11
7	Synopsys 3DIC Compiler Capability Assessment	11
7.1	Division of responsibility	13
7.2	Suggested sequencing	13
8	Risks and Open Items	14
9	Appendix: Source References	15
9.1	RTL	15
9.2	Synopsys documentation	16
9.3	Reproduction	16

1 Scope and Methodology

1.1 Objective

The question addressed is where to place the cut in a two-tier 3D implementation of TensorPool. The starting hypothesis, set by the architecture team, is that the second tier should not be used to fold the existing 2D floorplan (Groups stacked on Groups), but to relocate a level of the interconnect hierarchy — moving the complete logic, including routers, arbitration, registers and pipeline stages, not merely the wires.

The analysis covers four partitioning strategies, evaluates each against the actual RTL, and determines which parts of the flow can be handled by Synopsys 3DIC Compiler and which must be made explicit in the source.

1.2 Method

All structural statements in this report are derived from the RTL in `hardware/src/` on the `tensorpool` branch. All parameter values and signal widths were obtained by generating the Bender compile script for `config=tensorpool` and elaborating `mempool_pkg` in QuestaSim; none of the widths quoted here are hand-derived. All statements about Synopsys tool capability were verified against the local documentation set and the command reference, and no command is cited on the basis of its name alone.

Source	Version / location
RTL	<code>hardware/src/</code> , branch <code>tensorpool</code> , HEAD <code>d2df2897</code>
Configuration	<code>config/tensorpool.mk</code> , <code>config/config.mk</code>
Elaboration	QuestaSim 10.7b_1, Bender 0.32.1
3DIC Compiler User Guide	X-2025.06 and Y-2026.03, 258 / 269 pp.
3DIC Compiler Tool Commands	Y-2026.03, 5361 pp.
3DIC Compiler brochure	November 2025 revision

1.3 Configuration under analysis

TensorPool selects the three-level hierarchy. `hardware/Makefile` adds `-DTERAP00L` whenever `num_sub_groups_per_group != 1`, so TensorPool takes the ``ifdef TERAP00L` branch of `mempool_cluster.sv` and `mempool_group.sv`, which introduces the SubGroup level between Group and Tile. The MemPool and MinPool flavours take the two-level branch and have a different hierarchy, different widths and different interconnect counts; results in this report do not transfer to them.

Parameter	Value	Parameter	Value
Cores	256	Banks per tile	32
Groups	4	Total L1 banks	2048
SubGroups per Group	4	Bank size	2 KiB
SubGroups	16	Total L1 (TCDM)	4 MiB
Tiles per SubGroup	4	Banking factor	8
Tiles	64	Superbanks per tile	2
Cores per tile	4	RedMule instances	16 (8x32)
AXI masters per SubGroup	1	RedMule TCDM ports	16 x 32 bit
DMA backends per Group	4	Remote-Group latency	9 cycles
L2 size / banks	4 MiB / 4	AXI data width	512 bit

2 TensorPool RTL Hierarchy and Communication Structure

2.1 Where the crossbars actually are

The single most consequential structural fact for 3D partitioning is that the arbitration logic of the TCDM network is not located where the module names suggest.

- **The cluster top level contains no crossbar.** In the `TERAP00L` branch, `mempool_cluster.sv` contains only the DMA distribution mid-ends, AXI cuts, 192 sets of pipeline registers (lines 181–244) and a block of pure XOR-permutation `assign` statements (lines 395–408). What is informally called “the top-level interconnect” is physically a bundle of wires spanning the die, plus a register rank.
- **The inter-Group crossbar sits inside the source Group.** `mempool_group.sv:431–530` instantiates, for each remote Group `r`, one `burst_variable_latency_interconnect` configured as a fully connected crossbar with `NumIn = NumOut = 16`. With four Groups this gives **twelve 16x16 crossbars** in total. A request is routed to the destination Tile of the remote Group already on the sending side; the receiving Group performs no further arbitration and injects the request directly into the target Tile’s local crossbar.
- **The inter-SubGroup network at Group level is also only wiring.** `mempool_group.sv:413–425` is a second XOR-permutation block. The actual SubGroup-to-SubGroup crossbars are inside `mempool_sub_group.sv:346–443`: sixteen SubGroups x three remote ports = **48 crossbars of 4x4**.
- **Tile-level arbitration uses `stream_xbar`**, `mempool_tile.sv:855` and `:877`, 27 inputs to 32 banks for requests and 32 to 27 for responses. The 27 initiators are 20 local ports (four cores plus sixteen RedMule ports) and seven remote ports.

CONSEQUENCE

“Moving the top-level interconnect to the top tier” means, in RTL terms, relocating the `gen_remote_interco` generate block of `mempool_group` together with the cluster-level pipeline registers. `mempool_cluster` itself is already an empty shell and does not need to be moved.

2.2 Hierarchy tree

<pre> mempool_system +- mempool_cluster +- idma_split_midend + idma_distributed_midend +- 192 x 4 pipeline register sets (~145k FF) +- XOR permutation assigns (no logic) +- 4 x mempool_group +- 48 x 2 spill_register (~24k FF per Group) +- 3 x burst_variable_latency_interconnect LIC 16x16 <- inter-GROUP crossbar +- XOR permutation assigns (inter-SubGroup) +- idma_distributed_midend + 4 x axi_cut +- 4 x mempool_sub_group +- 4 x mempool_tile +- 4 x mempool_cc (Snitch + FPU + IPU) +- 1 x redmule_top 8x32 (tile 0 only) +- snitch_icache +- stream_xbar 27->32 / 32->27 +- 2 x tcdm_wide_narrow_mux (superbank) +- 32 x [tcdm_adapter + tc_sram 512x32] +- 1 x LIC 4x4 (SubGroup local) +- 3 x LIC 4x4 <- inter-SUBGROUP crossbar +- axi_hier_interco + snitch_read_only_cache +- 1 x axi_dma_backend + axi_xbar +- L2: 4 banks (4 MiB) + axi_xbar + ctrl_registers + bootrom </pre>	<pre> mempool_system.sv mempool_cluster.sv <- cluster top :87-126 :181-244 inter-Group pipeline :395-408 "top-level interconnect" :273-389 group.sv:202-231 latency = 9 group.sv:431-530 group.sv:413-425 group.sv:534-597 group.sv:273-407 sub_group.sv:151-238 tile.sv:197-283 tile.sv:1081-1124 tile.sv:286-330 tile.sv:855 / :877 tile.sv:443-478 tile.sv:635-687 L1 banks sub_group.sv:298-313 sub_group.sv:346-443 sub_group.sv:478 8 KiB R0\$ sub_group.sv:532 </pre>
--	---

2.3 Interconnect inventory

Level	Module and line	Count	Configuration	Wire length
Tile local	mempool_tile.sv:855 / :877	64 x 2	stream_xbar 27→32 / 32→27	intra-tile
SubGroup local	mempool_sub_group.sv:298	6	LIC 4x4	intra-SubGroup
Inter-SubGroup	mempool_sub_group.sv:346	8	LIC 4x4, 1 spill stage	intra-Group
Inter-Group	mempool_group.sv:431	12	LIC 16x16 , 1 spill stage	full die span
Cluster	mempool_cluster.sv:395	0	XOR permutation only	full die span

2.4 Interface widths

All widths below were read out of an elaborated `mempool_pkg` under `config=tensorpool`.

Type	Bits	Composition
tcdm_master_req_t	108	payload 47 + wen 1 + be 4 + tcdm_addr 18 + burst 38
tcdm_slave_req_t	108	payload 47 + wen 1 + be 4 + tile_addr 14 + tile_id 4 + burst 38
tcdm_master_resp_t	144	payload 47 + grouped burst response 97
tcdm_slave_resp_t	148	payload 47 + tile_id 4 + grouped burst response 97
tcdm_payload_t	47	meta_id 6 + core_id 5 + amo 4 + data 32
burst_t / burst_gresp_t	38 / 97	BurstLen 16, request grouping 2, response grouping 4
axi_tile_req_t / _resp_t	717 / 528	AXI data width 512
tcdm_dma_req_t / _resp_t	606 / 527	wide DMA port, 16 words
dma_req_t	113	iDMA burst descriptor
ro_cache_ctrl_t	258	enable, flush, 4 address ranges

The request path is **width-preserving across the inter-Group crossbar**: 108 bits enter as `tcdm_master_req_t` and 108 bits leave as `tcdm_slave_req_t`, because the 18-bit TCDM address is replaced by a 14-bit tile address plus a 4-bit tile identifier. On the response path 148 bits enter and 144 bits leave. This property is what makes the crossbar boundary essentially free as a tier cut, and is used in Section 4.

2.5 Existing pipelining on the inter-Group path

Two register banks already sit on the inter-Group path:

- `mempool_cluster.sv:181–244`: for each of the $4 \times 3 \times 4 \times 4 = 192$ channels, one `spill_register` on the master request, one `fall_through_register` on the master response, one `fall_through_register` on the slave request and one `spill_register` on the slave response. Approximately **145,000 flip-flops**.
- `mempool_group.sv:202–231`, enabled when `RemoteGroupLatencyCycle == 9`: 48 channels per Group, two `spill_register` instances each. Approximately **97,000 flip-flops** across four Groups.

`config/tensorpool.mk` sets `remote_group_latency_cycles = 9`, whereas `config/terapool.mk` sets `7`. TensorPool therefore deliberately added one pipeline stage per direction relative to TeraPool. Taken together, roughly **242,000 flip-flops** exist for the sole purpose of breaking the long inter-Group wires into multiple cycles. This is direct evidence that the inter-Group path is the 2D timing bottleneck, and it means a die-to-die hop can be inserted between existing registers without adding a pipeline stage.

CAVEAT ON REGISTER REMOVAL

The cluster-level registers are not purely a timing measure. The source comment states that they “break dependencies between request and response, establishing a correct valid/ready handshake”. At least one register in each direction is required for functional correctness; they cannot be removed wholesale. See Section 6.4.

2.6 AXI, DMA and L2 path

A second, largely independent network carries instruction fetch, L2 traffic and DMA. Most of its logic is at SubGroup level: each SubGroup instantiates one `axi_hier_interco` containing one 8 KiB `snitch_read_only_cache` (`mempool_sub_group.sv:478`), one `axi_dma_backend` with a 1-to-2 `axi_xbar` (:532), and exposes one AXI master port. Group level adds only four `axi_cut` instances and an iDMA distribution mid-end. The sixteen AXI master ports of the cluster connect in `mempool_system.sv` to a 4-bank, 4 MiB L2, an AXI crossbar, the control registers and the boot ROM. This path is latency-tolerant by construction and is relevant to the area-balance discussion in Section 5.3.

3 Candidate Tier Partitions

3.1 Tier-crossing signal counts

Each figure below counts payload bits plus valid and ready for every channel that would cross the tier boundary, using the widths of Section 2.4. Clock, reset, test and configuration signals are included in the Group-port figure and excluded elsewhere, where they are negligible.

Cut location	Option	Per unit	Total	Timing character
Below the inter-Group crossbar (SubGroup remote-Group ports)	C	24,768 / Group	99,072	registered both sides
As C, plus below the inter-Sub-Group crossbar	C+	+6,144 / SubGroup	197,376	one spill stage present
<code>mempool_group</code> ports as they exist today (Group-on-Group fold)	B	30,003 / Group	120,012	registered both sides
<code>tc_sram</code> pins	A / D	79 / bank	161,792	combinational, 1 cycle
Above <code>tcdm_adapter</code>	A / D	130 / bank	266,240	valid/ready, no pipeline
Complete tile memory subsystem	A	8,157 / tile	522,048	valid/ready, no pipeline
Cluster AXI ports (L2 to top tier)	—	1,245 / port	19,920	AXI, fully pipelined

Of the 120,012 signals at the existing Group boundary, 98,304 are TCDM; the remainder is AXI, DMA, wake-up and cache configuration.

KEY RESULT

The 99,072 vertical connections of Option C are within 0.8% of the 98,304 wires that already have to be routed horizontally across the die at the Group boundary today. Because the crossbar is width-preserving, relocating it does not increase the number of boundary signals; it converts millimetre-scale horizontal routing into micrometre-scale vertical bonds.

3.2 Option A — SubGroup-level folding

Partition each SubGroup across the two tiers, typically by placing the L1 memories on the top tier and keeping the cores on the bottom tier.

There is no RTL boundary to cut at. The banks are instantiated inline in the `gen_banks` generate block of `mempool_tile.sv:635–687`; no `mempool_tile_mem` module exists. More seriously, the `tcdm_adapter-to-tc_sram` path is a hard single-cycle loop: `out_req_o` is asserted and `out_rdata_i` must be valid in the following cycle. Crossing the tier boundary there adds latency to **every** L1 access, including the local ones that the MemPool memory model guarantees at one cycle. Vertical connection counts range from 161,792 to 522,048 depending on exactly where the cut is taken.

3.3 Option B — Group-level folding

Keep the SubGroups and the interconnect intact and fold whole Groups onto the second tier. This aligns perfectly with the existing RTL: `mempool_group` is already hardened as a macro in the 2D flow (see the `PostLayoutGr` hook at `mempool_pkg.sv:265` and `mempool_cluster.sv:275`), and the boundary already carries registers on both sides.

This is the option implemented on the existing `mbertuletti/3d` branch, where `hardware/src/terapool_top_die.sv` places Groups 2 and 3 on the upper die and Groups 0 and 1 on the lower die. It is a sound bring-up vehicle for the 3D flow, but it does not address the stated objective: the interconnect stays on the compute tier, the inter-Group wires still have to cross each die, and the routing channels between Groups are unchanged.

3.4 Option C — Interconnect-centric partition

Keep the compute-dominated SubGroups on the bottom tier and move the Group-level interconnect to the top tier. In RTL terms this relocates the twelve `burst_variable_latency_interconnect` instances of `mempool_group.sv:431–530` together with the cluster-level pipeline registers.

The cost is 99,072 vertical connections. No pipeline stage is added, because registers already exist on both sides of the cut. The bottom tier retains only intra-Group routing, so the routing channels that currently separate the four Groups can be largely eliminated, and the top tier provides a complete, otherwise unused metal stack for the inter-Group network.

The C+ variant additionally relocates the 48 inter-SubGroup 4x4 crossbars of `mempool_sub_group.sv:346–443`, bringing the total to 197,376 vertical connections and reducing each SubGroup to a purely local compute island. This variant is attractive for a second reason: the `tcdm_sg_*` ports of `mempool_sub_group.sv:41–53` already exist, making it the only cut in this study that requires no new module ports.

3.5 Option D — Memory plus interconnect partition

Move both the L1 memories and the higher-level interconnect to the top tier. This combines the benefit of C with the cost of A. Its motivation is area balance rather than routing: with only the interconnect on the top tier, that tier is sparsely utilised. Section 5 argues that the area-balance problem has cheaper solutions.

3.6 Comparison

Criterion	A	B	C / C+	D
Alignment with existing RTL hierarchy	none	exact	generate block must be extracted	none
Vertical connection count	162k–522k	120k	99k / 197k	260k+
Additional pipeline stages required	yes	no	no	yes
Relief of bottom-tier routing congestion	negligible	none	substantial	substantial
Group placement density improvement	none	none	yes	yes
Headroom for bandwidth increase	none	none	yes	yes
Top-tier area utilisation	high	high	low (needs filling)	high
Physical-design complexity	very high	low	moderate	very high
Impact on programming model	severe	none	none	severe

4 Recommended Partition

The recommendation is **Option C as the first step, with C+ as a follow-on**. Four pieces of evidence from the RTL support this.

The two interconnect levels cost the same but differ by an order of magnitude in wire length. Both the inter-Group and the inter-SubGroup networks total 64 tiles x 3 remote ports x 512 bits = 98,304 signals, because `NumGroups` and `NumSubGroupsPerGroup` are both 4. Since the vertical cost is identical, the level whose wires are longer should be relocated first. Inter-Group wires span the full die; inter-SubGroup wires stay within a Group.

The inter-Group path is already saturated with pipeline registers. As shown in Section 2.5, approximately 242,000 flip-flops exist solely to break these wires, and TensorPool raised the remote-Group latency from TeraPool’s 7 cycles to 9 in order to close timing. A die-to-die hop inserted between two of those registers is free.

The crossbar is width-preserving, so cutting below it costs nothing extra. 24,768 signals per Group below the crossbar versus 24,576 at the Group port — a difference of 0.8%.

The cluster level is empty, so relocating the crossbars breaks no module. Moving the twelve crossbars up turns `mempool_cluster` from a shell wrapped around a wire bundle into an actual top-level network-on-chip, which is also the more defensible architectural description.

4.1 Bond density feasibility

Bonding technology	Pitch	Density	Area for 99k / 197k signal bonds
Hybrid bonding, aggressive	1 μm	$10^6 / \text{mm}^2$	0.10 / 0.20 mm^2
Hybrid bonding, relaxed	4 μm	$6.3 \times 10^4 / \text{mm}^2$	1.6 / 3.2 mm^2
Microbump	9 μm	$1.2 \times 10^4 / \text{mm}^2$	8.2 / 16.4 mm^2

Allowing an additional 50% for power and ground bonds, hybrid bonding at any realistic pitch is not a constraint. Microbump assembly would need to be checked against the actual die area.

4.2 Prior art in the repository

The branch `origin/mbertuletti/3d` already contains related work and should be reviewed before any implementation starts:

Commit	Subject	Relevance
<code>043eab77</code>	Move cluster registers within the Group	Directly reusable; makes the Group a registered-IO macro
<code>90682bb9</code>	Add upper die (<code>terapool_upper_die.sv</code>)	Implements Option B, not Option C
<code>072b3fae</code>	Rename “upper_die” to “top_die”	Naming convention to follow

Commit `043eab77` moves the four cluster-level register sets into `mempool_group`, giving the Group fully registered TCDM interfaces at constant total latency. That refactor is a prerequisite for a clean Option C cut and should be adopted rather than reinvented. The remaining two commits implement the Group-on-Group fold and are orthogonal to the partition proposed here.

5 Memory on the Top Tier Versus Interconnect on the Top Tier

5.1 The case against relocating L1

Vertical connection cost is 1.6x to 5x higher. With 2048 banks, the cheapest cut — directly at the `tc_sram` pins — requires 161,792 connections; cutting above `tcdm_adapter` requires 266,240; relocating the complete tile memory subsystem requires 522,048.

The timing consequence is structural. The bank path is a hard single-cycle loop. Crossing tiers there adds latency to every L1 access, local accesses included. MemPool’s defining property is one-cycle local and at most five-cycle remote L1 access; this partition removes it.

There is no module boundary to cut at. The banks are inline in a generate block; a new `mempool_tile_mem` module would have to be created from nothing.

The routing benefit is close to zero. L1 consists of 2048 small 2 KiB macros tightly interleaved with the tile logic. The tile-internal crossbar wires are already short. Relocating the memories trades area without relieving any congestion.

5.2 Where a memory partition could work

Two qualifications are worth recording, because they bound the argument rather than overturn it.

First, a die-to-die hybrid bond is not inherently a pipeline stage. Its parasitic load is comparable to a few hundred micrometres of metal. If the memory tier were floorplanned directly above its own tile, with bonds landing inside the tile footprint, the crossing itself could be combinational; the problem is then the horizontal routing to and from the bond array, not the bond. At a 2 μm pitch a 250 x 250 μm tile offers roughly 15,000 bond sites against a requirement of 2,528, so the density is available.

Second, the TCDM interconnect is explicitly built for variable target latency — the module is named `burst_variable_latency_interconnect`, responses carry initiator metadata and are reordered by ID in `tcdm_shim`. Adding latency to a **subset** of banks is therefore functionally supported. Moving one of the two superbanks per tile to the top tier would produce a non-uniform L1 with a fast and a slow half.

Both observations describe an **architectural** change, not a physical partition: they alter the memory model that software and the published performance results rely on. They should be evaluated as a separate study, not folded into the first 3D implementation.

5.3 Filling the top tier without moving L1

The area imbalance of Option C is real. Twelve 16x16 crossbars and roughly 242,000 flip-flops represent an estimated 10–20% of the design’s flip-flop count and considerably less of its area. Three remedies, in order of attractiveness:

1. **Relocate the L2 subsystem.** L2 is 4 MiB across four banks — the same capacity as the entire L1 — and sits behind AXI with tens of cycles of latency, so it is completely latency-tolerant. Cutting at the sixteen AXI master ports of `mempool_cluster` costs only **19,920 vertical connections**. This is by a wide margin the cheapest way to add memory area to the top tier, and it also moves the AXI system crossbar, control registers and boot ROM off the compute tier.
2. **Execute C+ and relocate the SubGroup AXI subsystem.** The 48 inter-SubGroup crossbars, the sixteen `axi_hier_interco` instances with their 8 KiB read-only caches, the sixteen DMA backends and their AXI crossbars are all system logic whose relocation does not touch the TCDM latency contract.
3. **Accept an asymmetric stack.** The top tier does not have to match the bottom tier in area, and it can use an older, cheaper process — 3DIC Compiler supports per-die technologies and scale factors (User Guide, Chapter 4). For a research vehicle a smaller interconnect die is a legitimate answer.

The resulting recommended top-tier content is: inter-Group crossbars, cluster pipeline registers, L2 with its AXI infrastructure, and optionally the inter-SubGroup crossbars and the SubGroup AXI subsystem.

Combined vertical cost: approximately 119,000 connections for the C-plus-L2 variant, or approximately 217,000 for the C++-plus-L2 variant.

5.4 Thermal considerations

Concentrating all compute logic on one tier is favourable for heat extraction: the power density is confined to a single plane rather than stacked, and in a face-to-face assembly the compute die can be placed on the side facing the heat sink, with the low-activity interconnect and L2 die on the far side. This is a significant secondary argument for Option C over any partition that splits compute across both tiers.

The assembly orientation must be verified rather than assumed, because it interacts with power delivery: whichever die is furthest from the C4 bumps needs TSVs for its power network. 3DIC Compiler provides `analyze_3d_thermal` with the built-in engine or RedHawk-SC Electrothermal (User Guide, Chapters 11–12) and `analyze_3d_rail` for EM/IR, and both can be run during feasibility exploration before any RTL is written.

6 Required RTL Restructuring

Changes are classified as (1) strictly required by the Synopsys flow, (2) not required but materially simplifying physical design, or (3) an architectural modification rather than a physical partition.

6.1 Option C

Change	Detail	Class
Two tier top-level modules	<code>tensorpool_bottom_tier</code> and <code>tensorpool_top_tier</code> , each a complete synthesisable design. The 3D top-level netlist must instantiate dies that are already implemented separately (User Guide, Chapter 7).	1
3D top-level netlist	<code>tensorpool_3d_top</code> , containing only the two die instances and an optional empty interposer placeholder. This is the input to <code>create_3d_top_design</code> and <code>read_verilog</code> .	1
Every die-to-die signal must be a port	Bumps and bond pads attach to ports; internal nets cannot be assigned.	1
Per-die clock and reset ports	Today a single <code>clk_i</code> fans out to the whole cluster. Each die needs its own clock entry for independent CTS, and die-to-die skew must be constrained in multi-die STA.	1
Move cluster registers into the Group	Follows <code>043eab77</code> . Gives the Group registered TCDM interfaces and places one register rank on the bottom side of the cut at unchanged total latency.	2
Extract <code>gen_remote_interco</code> into a module	<code>mempool_group.sv:431–530</code> is an inline generate block using Group-internal signals. Extracting it into <code>tensorpool_inter_group_interco</code> is mechanical; without it the twelve instances can only be lifted by netlist-level <code>group / ungroup</code> operations, which is fragile.	2
Explicit die-to-die interface bundle	A named packed type plus a flat, sorted port naming scheme such as <code>d2d_g<g>_sg<sg>_t<t>_*</code> .	2

Change	Detail	Class
	<code>derive_3d_connections</code> connects die-internal nets to top-level ports by name , so a regular naming scheme makes bump planning fully scriptable and CSV-driven.	
Per-die scan chains	<code>scan_data_i</code> / <code>scan_data_o</code> are currently left unconnected (for example <code>mempool_cluster.sv:280–281</code>). A manufacturable design needs per-die test access under IEEE 1838.	2
Change <code>RemoteGroupLatencyCycle</code>	Changing 9 to 7 removes the Group-level spill registers and two cycles of remote latency. Visible to software and to the published performance model.	3
Widen the inter-Group network	Changing <code>NumIn</code> / <code>NumOut</code> , the burst grouping factors <code>GROUP_REQ</code> / <code>GROUP_RSP</code> , or the number of remote ports per tile.	3

6.2 Option C+ (additional)

Extract the three 4x4 crossbars of `mempool_sub_group.sv:346–443` into a module and relocate them to the top tier. The `tcdm_sg_*` ports of `mempool_sub_group.sv:41–53` then become die-to-die ports directly; no new ports are needed. `mempool_group` degenerates into a pure container and could be removed as an RTL level, although keeping it is harmless and preserves the existing hardening flow.

6.3 Options A and D (additional)

Create a `mempool_tile_mem` module containing `gen_banks`, the `tcdm_wide_narrow_mux` instances and possibly both `stream_xbar` instances (**class 1**); insert pipeline stages on the bank path and modify the `tcdm_adapter` handshake, which currently assumes the SRAM responds in the next cycle (**class 3**); and update every latency assumption in `software/runtime` (**class 3**).

6.4 Reducing the pipeline depth

A shorter physical interconnect permits a shorter pipeline, and the parameterisation already supports part of this. `RemoteGroupLatencyCycle` accepts 7, 9 or 11; at 7 the Group-level register banks of `mempool_group.sv:148–238` are not instantiated at all. Moving from 9 to 7 is therefore a single configuration change that removes roughly 97,000 flip-flops and two cycles of remote-access latency, and it is the natural first experiment once the 3D floorplan shows the shortened wires.

Going below 7 is not a configuration change. The remaining registers are the cluster-level spill/fall-through pairs, which exist to break the request-to-response dependency and guarantee a correct valid/ready handshake. Removing them requires microarchitectural work on the handshake, not just deletion, and must be validated against deadlock.

7 Synopsys 3DIC Compiler Capability Assessment

3DIC Compiler is an assembly, analysis and verification platform, not an RTL partitioning tool. Its documented flow assumes that each die is an independently implemented design — produced in Fusion Compiler — which is then instantiated in a 3D top-level netlist, placed with `create_die` and `set_cell_location -z_offset`, connected through bumps and bond pads, checked and analysed. No command exists that splits one netlist into two dies.

Capability sought	Finding	Source
Die / tier partitioning	Not present. The term does not appear anywhere in the User Guide in the sense of die assignment.	User Guide, full text
Hierarchy assignment to dies	Not present. Every <code>*_3d_*</code> command concerns bumps, TSVs, stacking, routing, checking, thermal analysis or AI-driven exploration.	Command reference, full command list
Hierarchy push / pull	<code>push_down_objects</code> and <code>pop_up_objects</code> operate on pseudo bump regions in the physical hierarchy, not on logical hierarchy.	User Guide, Ch. 5
<code>auto_partition_design</code>	Exists, but is the conventional 2D hierarchical-partitioning flow: it regroups existing logical hierarchies into virtual groups under cell-count and pin-count constraints. Output is hierarchy within one block, not two dies, and its quality is bounded by the RTL hierarchy it starts from.	Command reference, p. 256
<code>partition_block</code>	Clips a physical region into a standalone block. The manual states it “is always a valid design, though not always a complete sub-circuit” and “makes no additional edits to clean up incomplete states such as dangling nets, missing drivers”. An analysis aid, not an implementation path.	Command reference, p. 2844
Blocks spanning dies	Not supported. Each die is a separate NDM block; the top level only instantiates and places them.	User Guide, Ch. 7
3D interface definition	Supported, at the physical level. <code>derive_3d_interface</code> derives bump, TSV and bond-pad objects from existing ones; <code>create_3d_virtual_blocks</code> creates virtual interface blocks for	User Guide, Ch. 6

Capability sought	Finding	Source
Vertical interconnect assignment	early exploration; <code>derive_3d_connections</code> connects die-internal nets to top-level ports by name. Strong. <code>create_bump_region</code>, <code>create_3d_mirror_bumps</code>, <code>assign_3d_interchip_nets</code> (shortest-wirelength as- signment), <code>propagate_3d_connections</code>, <code>commit_pseudo_bumps</code>; lo- cations and connections can be driven from CSV via <code>read_design_io</code> and <code>read_block_connection_file</code>	User Guide, Ch. 5 and 7
Cross-die optimisation	Not logical optimisa- tion. 3DSO.ai searches configuration space (bump density, place- ment, thermal parameters); <code>analyze_3d_thermal</code> and <code>analyze_3d_rail</code> cover multiphysics; cross-die timing is StarRC plus PrimeTime hyperscale. There is no cross-die syn- thesis or retiming.	Brochure; User Guide Ch. 11–14

7.1 Division of responsibility

Must be expressed in RTL

- Split into two independently implementable die top modules
- Die-to-die signals as ports on both modules
- The 3D top-level netlist
- Per-die clock, reset and scan
- Die-to-die port naming convention
- Any change in pipeline depth

Handled by 3DIC Compiler

- Bump, bond-pad and TSV array creation and planning
- Bump mirroring between dies and legality checking
- Shortest-wirelength signal-to-bump assignment
- Die placement, z-ordering and overlap checking
- More than 60 classes of 3D design rule check
- Thermal, EM/IR and SI analysis; feasibility exploration
- Per-die technologies and scale factors

7.2 Suggested sequencing

Feasibility analysis does not require a PDK and does not require any RTL change. Before committing to the restructuring of Section 6, `create_3d_virtual_blocks` and the feasibility flow (User Guide, Chapter 12) can be used to establish the bond area required for 99,000 to 197,000 connections at the candidate pitch, and to obtain first thermal and IR estimates. Only after that should the RTL work begin.

8 Risks and Open Items

Item	Status
Area split by synthesis	Not measured. The area statements in this report are inferred from flip-flop counts. A hierarchical synthesis run of <code>mempool_group</code> with <code>set_dont_touch</code> on <code>gen_remote_interco</code> , the SubGroups, the Tiles and the L1 is needed before the top-tier die size and process can be chosen.
Power delivery	Not analysed. The die furthest from the C4 bumps requires a TSV-based power network; bond budget must be re-checked with PG bonds included.
Bond pitch and yield	Process-dependent; requires the target foundry stack.
2D reference QoR	The published 2D floorplan results should be re-measured on the same RTL revision to give a clean baseline for the 3D comparison.
Interaction with <code>mbertuletti/3d</code>	That branch implements Option B and carries a large number of unrelated changes (RedMulE parametrisation, tile merge, address slicer, software). Only commit <code>043eab77</code> should be adopted directly.
Non-TERAPOOL configurations	MemPool and MinPool use the two-level branch, which contains its own <code>gen_remote_interco</code> at <code>mempool_group.sv:829</code> . Any refactor must either preserve that path unchanged or apply the same treatment consistently.

9 Appendix: Source References

9.1 RTL

Statement	Reference
TensorPool selects the three-level TERAP00L hierarchy	hardware/Makefile:106-108 ; config/tensorpool.mk
Cluster top level has no crossbar	mempool_cluster.sv:181-244 , :395-408
Inter-Group crossbar: 12 x LIC 16x16	mempool_group.sv:431-530 (instantiation at :486)
Inter-SubGroup connection at Group level is wiring only	mempool_group.sv:413-425
Inter-SubGroup crossbar: 48 x LIC 4x4	mempool_sub_group.sv:346-443 (instantiation at :399)
tcdm_sg_* ports already exist	mempool_sub_group.sv:41-53
Inter-Group pipeline, approximately 242k flip-flops	mempool_cluster.sv:181-244 ; mempool_group.sv:202-231
Remote-Group latency raised from 7 to 9	config/tensorpool.mk versus config/terapool.mk
L1 banks are inline in the tile, no module boundary	mempool_tile.sv:635-687
SRAM path is a hard single-cycle loop	mempool_tile.sv:650-687
Tile crossbars 27→32 and 32→27	mempool_tile.sv:855 , :877
RedMulE on tile 0 of each SubGroup, 16 ports	mempool_tile.sv:1081-1152 ; mempool_pkg.sv:283-293
SubGroup and Group are already hardened as macros	mempool_pkg.sv:264-265 ; mempool_cluster.sv:275 ; mempool_group.sv:291
SubGroup AXI sub-system and read-only cache	mempool_sub_group.sv:478 , :532
L2, AXI crossbar, control registers, boot ROM	mempool_system.sv:136 onwards
All parameter and width values	QuestaSim elaboration of mempool_pkg , config=tensorpool

9.2 Synopsys documentation

Statement	Reference
3D flow requires independently implemented dies and a 3D top-level netlist	User Guide Ch. 3 (Fig. 9, 10), Ch. 5, Ch. 7
No die-partitioning command exists	User Guide, full text; Command reference, full command list
<code>auto_partition_design</code> is 2D hierarchical partitioning	Command reference, p. 256
<code>partition_block</code> clips a physical region and does not guarantee a complete sub-circuit	Command reference, p. 2844
<code>push_down_objects</code> / <code>pop_up_objects</code> apply to pseudo bump regions	User Guide Ch. 5, “Bump Planning Flow”
Bump, TSV, mirroring, assignment and checking commands	User Guide Ch. 5, 6, 7
Thermal, feasibility, EM/IR analysis	User Guide Ch. 11, 12, 13
Hybrid bonding, face-to-face and face-to-back, direct 3D stacking, heterogeneous processes	Brochure; User Guide Ch. 4
Multi-die STA and DFT	Brochure (StarRC and PrimeTime hyperscale; IEEE 1838, TestMAX Manager)

9.3 Reproduction

The parameter and width values in Sections 2.3 and 2.4 can be reproduced by generating the Bender compile script with the full `config=tenorpool` macro set, compiling it in QuestaSim, and elaborating a module that imports `mempool_pkg` and prints `$bits` of each type. The tier-crossing counts of Section 3.1 follow from those widths by summing payload plus valid plus ready over the channel counts given in Sections 2.1 and 2.3.

End of report. No RTL was modified in the preparation of this analysis.